

qcode

User Manual

A terminal-first AI coding assistant.
This guide shows how to use qcode day to day.

Version dev - September 30, 2026

Usage guide for people using qcode.

No setup internals or code architecture inside.

In this manual: install once, tour the screen, try practical use cases,
clarifying questions, switch models, use agents, plan, skills, memory,
web tools, sessions, browser control, and settings.

How to use this manual

Read chapters 1-3 to start. Keep chapters 4 and 15 nearby as a command reference. The Use cases pages after chapter 3 offer complete example workflows. Chapters 5-12 are task guides you can read when you need them: models, agents, planning, skills, memory, web, sessions, and phone/browser control.

- Words in ``code style`` are things you type, for example ``/model`` or ``qcode --demo``.
- Figures are illustrative example screens. Your colors and sizes may differ slightly.
- The prompt line shows ``>`` when idle, ``(Plan)>`` in Plan mode, ``(Skill plan)>`` in Skill Plan mode, and ``(Steer)>`` while busy.
- Esc goes back one level. Ctrl+C cancels without applying changes.

Tip

Try everything first with: `qcode --demo "show me how qcode works"`. Nothing is installed and your files are untouched.

1. Getting started

Install and first run

qcode is one program you run in your terminal. Build it once, then start it in any project folder.

```
# Build from source (needs Go 1.22+)
make build
./bin/qcode
```

Pick the assistant you want to use. The two common choices are a local model (Ollama) or a hosted model (OpenAI or OpenCode Go).

```
# Local model (default provider)
ollama pull qwen2.5-coder:7b
qcode --model qwen2.5-coder:7b

# Hosted model
export OPENAI_API_KEY=...
qcode --provider openai --model gpt-5

# No setup tour
qcode --demo "show me how qcode works"
```

Tip

If you only want to learn the keys and screens, start with --demo. It runs a scripted tour with fake tools and queued prompts.

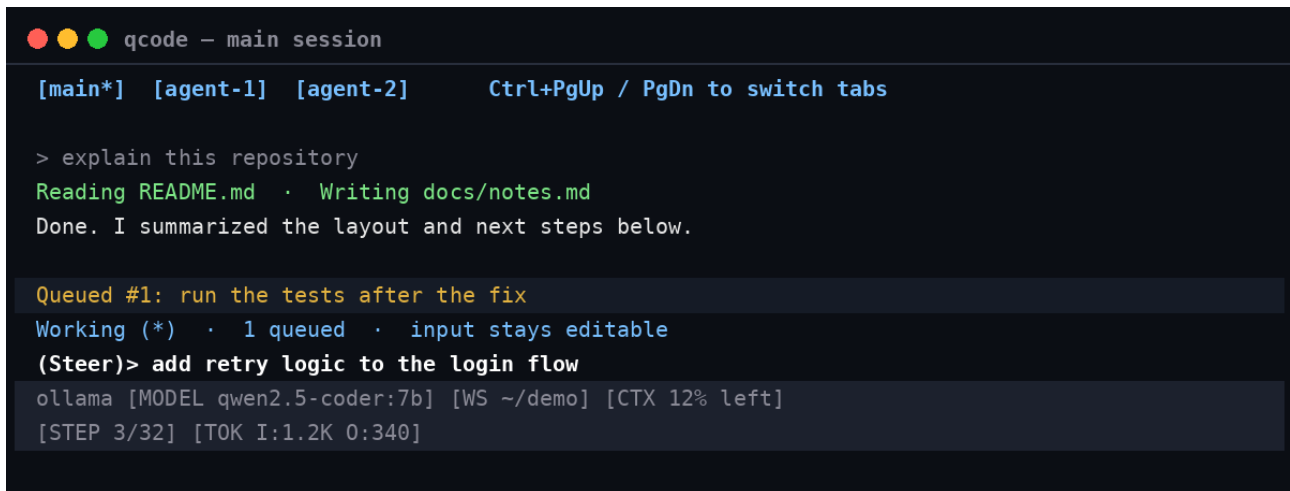
Update

Use the shell command below to install the latest release. It is a qcode CLI command, not a slash command.

```
qcode update
qcode update --arch arm64
```

2. Touring the terminal

qcode fills the terminal but keeps your normal scrollbar. Tabs sit at the top, the conversation fills the middle, and the prompt plus status bar stay pinned at the bottom.



```

qcode - main session

[main*] [agent-1] [agent-2]      Ctrl+PgUp / PgDn to switch tabs

> explain this repository
Reading README.md · Writing docs/notes.md
Done. I summarized the layout and next steps below.

Queued #1: run the tests after the fix
Working (*) · 1 queued · input stays editable
(Steer)> add retry logic to the login flow
ollama [MODEL qwen2.5-coder:7b] [WS ~/demo] [CTX 12% left]
[STEP 3/32] [TOK I:1.2K O:340]
  
```

Figure 1: Main screen. Tabs on top, transcript in the middle, queue notice, editable prompt, and status bar at the bottom.

What you see

- Tabs: `main` is always there. Extra agents appear as new tabs (up to 20 total).
- Transcript: streamed answers with Markdown, tables, and tool activity like `Reading ...` or `Writing ...`.
- Pending area: shows the current steer separately from FIFO tasks.
- Prompt: `>` idle, `(Plan)>` planning, `(Skill plan)>` skill design, `(Steer)>` working. You can keep typing while work runs.
- Status bar: provider plus [MODEL name], [WS folder], [CTX % left], [STEP n/max], [TOK I/O], plus [THINK level] when set, [MODE PLAN] / [MODE INTERACTIVE] only when active, and [REMOTE] only while browser control runs. No MODE badge appears for normal work. Hide parts with `/statusline`.

Keys you will use daily

- Type `/` to see up to five matching commands. Type more to filter, Tab to complete the first match, Esc to close.
- Left/Right edits input. At the bottom of output, Home/End move to input ends. Ctrl+Left/Right jumps by word; Ctrl+W deletes a word; Ctrl+A/E moves to input ends.
- PageUp/PageDown scrolls history with two overlapping rows of context, even while the agent works. Scrolling pauses the live view; PageDown to the bottom resumes it.
- When the view is above the bottom, Home/End jump to oldest/latest output. At the bottom they edit the prompt again. Browser text fields keep editing keys.
- Alt+Q opens pending work. Up/Down selects, PgUp/PgDn scrolls, Delete removes, and Esc closes.
- Ctrl+C cancels the running prompt. Queued prompts wait their turn.

- `NO_COLOR=1` disables colors; `QCODE_ASCII=1` uses ASCII glyphs.

3. Asking, queueing, and history

Type a request and press Enter. While one request runs, you can type the next one. Enter steers the current task; Tab queues a separate task. Steering waits for a response or tool, replaces pending steering, and withdraws unanswered interactions.

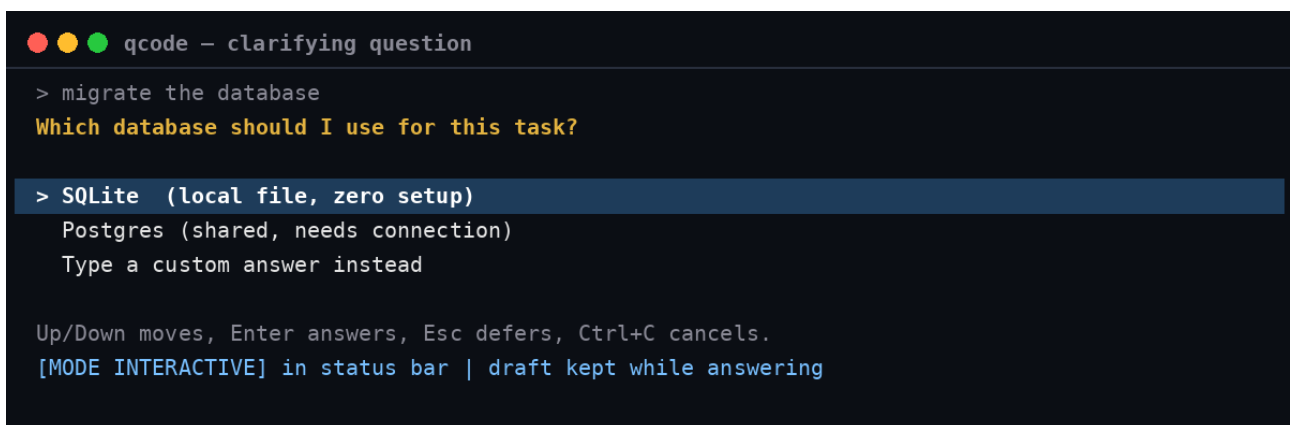
- One prompt runs at a time per agent. Other agents are not blocked.
- The tab and task line show how many prompts are waiting.
- Use ``/history`` to find an old prompt. Pick one to re-read its final answer.
- Use ``/new`` to clear the current conversation without restarting qcode.
- Use ``/compact`` when a long session feels slow. It shortens stored context.

Tip

Example flow: ask ``explain this repository``, then while it works queue ``run the tests after the fix``. Both complete in order without you waiting.

When qcode asks you back (`/interactive`)

Normal work can pause to ask one focused question when an ambiguity would otherwise need several broad searches. Turn this on per agent with ``/interactive on``, off with ``/interactive off``, or check the setting with ``/interactive``. It is off by default. Add ``interactive = true`` to `config.toml` to enable it for new terminal sessions.



```

qcode - clarifying question

> migrate the database
Which database should I use for this task?

> SQLite (local file, zero setup)
Postgres (shared, needs connection)
Type a custom answer instead

Up/Down moves, Enter answers, Esc defers, Ctrl+C cancels.
[MODE INTERACTIVE] in status bar | draft kept while answering
  
```

Figure 3: A clarifying question. Pick a suggestion or type your own. At most 3 questions per prompt.

- One question at a time, with suggested choices or your own custom answer. At most 3 distinct questions per submitted prompt.
- Your answer becomes context for that agent only. It is not auto-saved; use ``/learn`` to keep it.
- The status bar shows `[MODE INTERACTIVE]` while enabled (`[MODE INT]` when narrow). Your half-typed draft is saved and restored around the question.
- Esc defers a question and opens the composer; Esc returns to it. A steer withdraws unanswered interactions. Ctrl+C cancels active work. The browser can also answer.
- One-shot prompts, piped input, non-terminal runs, and `--json-events` never wait; qcode just proceeds with available context.

Tip

Turn it on when tasks are ambiguous (which database, which scope). Leave it off for strict hands-off runs.

Use cases: understand and fix

These are example requests, not built-in commands. Replace paths, commands, and symptoms with those from your project. Review the resulting files and command output before accepting a change.

Understand an unfamiliar repository

Goal: find the entry points and learn how to run the project before making changes.

1. Start qcode in the project folder and submit the example below.
2. Ask a follow-up about the part you need to change. Use /history to revisit the explanation.
3. Check the cited files and confirm the suggested commands against the project's README.

```
Explain this repository without changing files. Identify the
entry points, how to run it, and how to run its tests. Cite the
files you used, and list anything you could not confirm.
```

Result to look for: a file-grounded starting guide with clear run and test commands, plus open questions.

Fix a bug and queue the next step

Goal: reproduce a specific failure, make a focused fix, and check the result.

1. Use /interactive on if expected behavior needs clarification; then describe the failure and a reproduction command.
2. While the fix runs, press Tab to queue the follow-up below. It waits in the active agent's FIFO queue.
3. Review the diff with /diff and inspect test output. Ask for an explanation if the checks fail.

```
Fix the login timeout: retry once after a temporary network
failure, but never retry invalid credentials. Reproduce it
with the login tests before changing the handler.
```

```
# Submit while the first request is running:
Run the login tests after the fix and summarize the results.
```

Result to look for: a small diff and observed test results, with any remaining failure stated explicitly.

Use cases: plan and divide work

Plan a change before editing

Goal: settle scope and review the approach before implementation.

1. On an idle agent, enter `/plan` and describe the change below.
2. Answer design questions, then use `/plan show` to review the proposal. Request revisions if needed.
3. Use `/plan act` only when ready to implement. Review the diff and the planned checks afterward; `/plan off` leaves without implementing.

```
/plan
Plan pagination for the search results. Inspect the current
API and UI, identify compatibility concerns, and propose
acceptance checks before changing files.
/plan show
/plan act
```

Result to look for: an agreed plan followed by changes that meet its acceptance checks.

Use independent agents for implementation and investigation

Goal: keep a focused implementation moving while another tab investigates a separate question.

1. Give main the implementation task. Use `/agent new` to pick a model for a helper; rename its displayed ID with `/agent rename <id> review-tests`.
2. In the helper tab, ask for a read-only review of relevant tests and missing edge cases. Switch tabs with `Ctrl+PageUp/PageDown`.
3. Bring the helper's findings back to main, request the needed checks, and review the combined result.

```
# Main prompt:
Add the agreed pagination behavior to the search endpoint.

# Helper prompt:
Read the search tests without changing files. Identify
missing pagination edge cases and report the file locations.
```

Tip

Agents share the workspace. Give them distinct responsibilities; avoid asking two tabs to edit the same files at once. Each tab has its own history and queue.

Use cases: reuse and continue

Turn a repeated review into a skill

Goal: make a repeatable checklist available in future work.

1. Start `/skillplan` with the example below and answer questions about scope, inputs, and validation.
2. Review with `/skillplan show`; request edits until the draft describes your actual workflow.
3. Use `/skillplan create` to write the reviewed skill, then `/skill` to enable it before the next review.

```
/skillplan Create a database migration review checklist.  
Check rollback behavior, existing data compatibility,  
and validation commands. Report risks with file references.  
/skillplan show  
/skillplan create  
/skill
```

Result to look for: an enabled, reusable skill whose instructions and completion checks you have reviewed.

Resume work and share its outcome

Goal: continue a saved conversation and produce a readable handoff.

1. Return to the same project folder, start qcode, and use `/resume` to choose the saved session.
2. Use `/history` to find the last completed answer. Ask for current status and remaining work before continuing; interrupted tools are not rerun automatically.
3. Use `/export pretty` for a shareable HTML conversation, or `/export raw` when the handoff needs detailed activity.

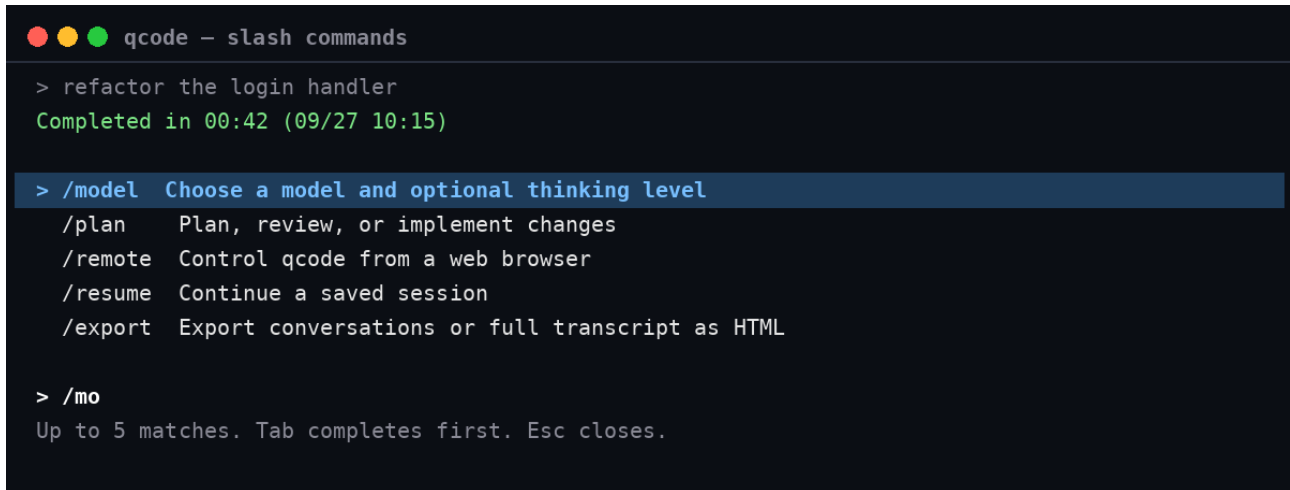
```
/resume  
Summarize the completed changes and remaining work. Verify  
the current files before continuing the unfinished task.  
/export pretty
```

Tip

To continue from a phone on a trusted network, use `/remote` and redeem the QR link. The browser controls the same live session; its `/export` downloads the HTML. Close the connection when finished.

4. Slash commands at a glance

Type ``/help`` to list commands or ``/help <name>`` for one command. The leading ``/`` is optional in the name. Agent, skill, learning, plan, remote, model, tool, and interactive commands are covered in their chapters below.



```

qcode - slash commands
> refactor the login handler
Completed in 00:42 (09/27 10:15)

> /model Choose a model and optional thinking level
/plan    Plan, review, or implement changes
/remote  Control qcode from a web browser
/resume  Continue a saved session
/export  Export conversations or full transcript as HTML

> /mo
Up to 5 matches. Tab completes first. Esc closes.

```

Figure 2: Type `/` to filter up to five commands. Tab completes the first match.

Command	What it does for you
<code>/help</code>	List commands or explain one command.
<code>/bash <cmd></code>	Run a shell command in the workspace.
<code>/new</code>	Start a fresh conversation for the active agent.
<code>/resume</code>	Continue a saved session (picker or session-id).
<code>/clear</code>	Clear the visible conversation and redraw the header.
<code>/history</code>	Browse completed prompts and responses.
<code>/diff [N]</code>	View more lines of saved diff N; omit N for latest.
<code>/verbose</code>	Show or hide detailed action traces.
<code>/maxsteps [N]</code>	Show or change the model-turn limit.
<code>/statusline</code>	Show, hide, or reset status bar parts.
<code>/theme</code>	Preview, apply, and save a terminal color theme.
<code>/compact</code>	Summarize old context to make room.
<code>/export</code>	Export conversations or full transcript as HTML.
<code>/quit, /exit</code>	Save the session and exit qcode.

Diff previews

When qcode writes or edits a file you see a short numbered 10-line preview with added and removed lines. Use ``/diff`` to expand the latest saved diff or ``/diff N`` to expand saved diff N, up to the 200-line safety limit. The model still receives plain text.

A terminal window with a dark background and light-colored text. The title bar shows three colored circles (red, yellow, green) followed by the text 'qcode - diff preview and export'. The terminal content shows a diff command being executed, which displays a code change in a syntax-highlighted format. The change involves a function call in a Python file. Below the diff, there is a message about the diff command's behavior. Finally, the user enters an export command, and the terminal shows the resulting HTML file name and a note about custom paths.

```
qcode - diff preview and export

Writing internal/app.py (+12 -3) · diff 1
@@ login handler @@
    context = load_session()
-   login_once(context)
+   login_with_retry(context, attempts=3)

/diff expands latest; /diff 3 expands saved diff 3 (to 200)

> /export pretty
Exported session to qcode-session-pretty-20260927-101500-
000000001.html (custom paths are not accepted)
```

Figure 4: Numbered change preview plus `/export pretty` saving a readable HTML copy.

Exports

- ``/export`` or ``/export pretty``: one tab per agent, prompt cards that reveal answers. Best for sharing and reading.
- ``/export raw``: full styled transcript including tool activity, thinking, errors, and diffs.
- Files are saved as ``qcode-session-<mode>-<timestamp>.html`` with subsecond precision. Custom output paths are not accepted.

5. Choosing models and providers

Use ``/model`` to switch the assistant for the current tab. The list shows model names from your current provider only, with no provider suffix. Type to search, move with Up/Down and PgUp/PgDn, Enter to pick. You can also run ``/model <id> [thinking]`` directly. Some models ask a second question for thinking level; Esc returns to the model list and Ctrl+C cancels.



```

qcode - /model picker

Select model (4/4) | Up/Down, PgUp/PgDn | Search:

> qwen2.5-coder:7b
  qwen3:8b
  gemma3
  gpt-oss:20b

List shows the current provider's model names only.
Thinking: off / low / medium / high / max (model decides list)

```

Figure 5: `/model picker`. Model names only; the list comes from the current provider. An optional thinking level follows (off/low/medium/high/max). Choices depend on the model.

Providers you can use

- Ollama (local default): pull a model first, then ``qcode --model qwen2.5-coder:7b``. Thinking uses the model's native think setting.
- OpenAI-compatible: ``export OPENAI_API_KEY=...`` then ``qcode --provider openai --model gpt-5``. Thinking uses reasoning effort where the model offers it.
- OpenCode Go: ``export QCODE_API_KEY=...`` then ``qcode --provider opencode-go --model kimi-k3``. Options vary per model.
- Flags can also come from `QCODE_PROVIDER`, `QCODE_MODEL`, `QCODE_THINKING`, `QCODE_BASE_URL`, `QCODE_API_KEY`.

```

qcode --provider ollama --model qwen3:8b --thinking high "reply with OK"
qcode --provider opencode-go --model deepseek-v4-flash --thinking high "reply with OK"

```

Tip

On ``main``, `/model` and `/maxsteps` remember your choice. Other tabs, demo, one-shot, and piped runs keep changes for that session only.

One-shot and demo

```

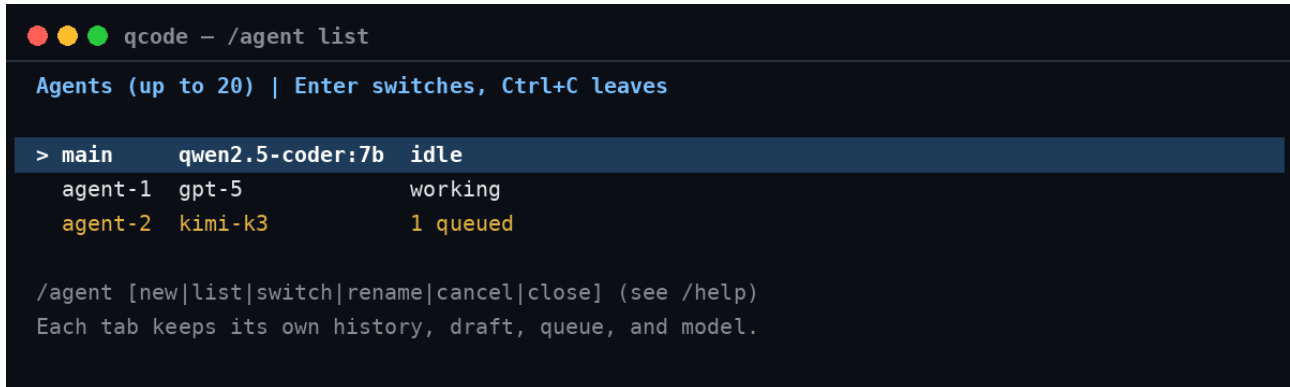
qcode "explain this repository"
qcode --json-events "run the tests" 2>events.jsonl
qcode --demo "show me how qcode works"

```

Pass a prompt to run once without the interactive screen. Assistant text goes to stdout and short progress events go to stderr. Demo mode needs no model and touches no files.

6. Working with multiple agents

Use extra agents to do independent jobs at the same time, for example one writing code while another reads tests. Each tab keeps its own history, draft, queue, model, and tool choices.



```

qcode - /agent list

Agents (up to 20) | Enter switches, Ctrl+C leaves

> main      qwen2.5-coder:7b  idle
agent-1     gpt-5             working
agent-2     kimi-k3           1 queued

/agent [new|list|switch|rename|cancel|close] (see /help)
Each tab keeps its own history, draft, queue, and model.

```

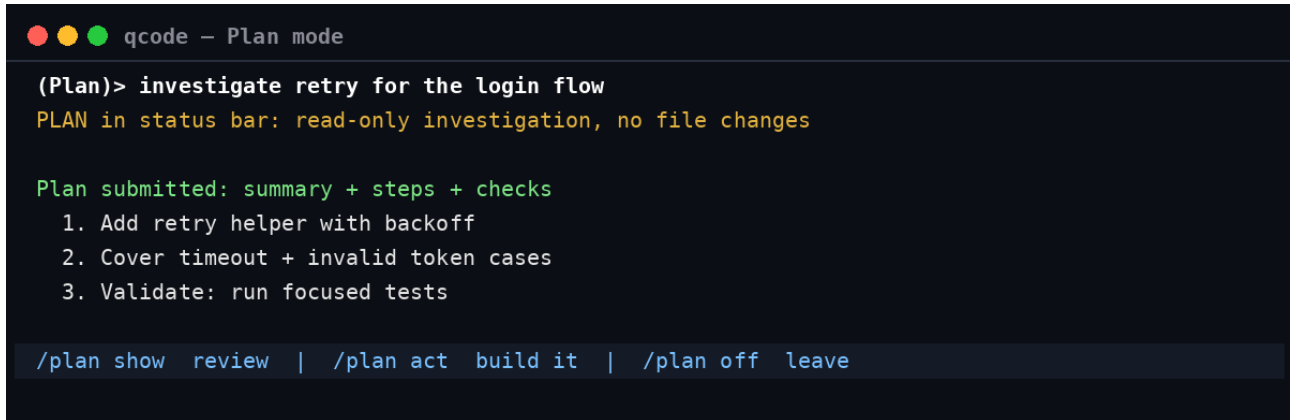
Figure 6: /agent list. Enter switches to the highlighted agent.

Command	What it does for you
/agent	Create an agent (same as /agent new).
/agent new [model]	Create an agent, optionally with a model ID.
/agent list	Pick and switch with Up/Down + Enter.
/agent switch <id>	Jump directly to one agent.
/agent rename <id> <n>	Give an agent a new display name.
/agent cancel <id>	Stop that agent's current work.
/agent close <id>	Close that agent tab (--yes skips confirm).

- Switch with Ctrl+PageUp/PageDown (Alt+, and Alt+. also work).
- Ask the main agent to split work and it can coordinate helpers; you can also create tabs yourself with /agent.
- Queues are per agent (up to 16 waiting each). A slow helper never blocks `main`.

7. Planning before coding

Plan mode investigates without changing files. Enter it while idle, let qcode study the code, answer any design questions, review the submitted plan, then build it.



```
qcode - Plan mode

(Plan)> investigate retry for the login flow
PLAN in status bar: read-only investigation, no file changes

Plan submitted: summary + steps + checks
  1. Add retry helper with backoff
  2. Cover timeout + invalid token cases
  3. Validate: run focused tests

/pplan show review | /plan act build it | /plan off leave
```

Figure 7: Plan mode. The prompt shows (Plan) and the status bar shows [MODE PLAN] until you act or leave.

1. Type ``/plan`` to enter. The status bar shows PLAN.
2. Describe the task. Answer focused questions if qcode asks (you can type a custom answer).
3. Type ``/plan show`` to re-read the latest plan any time.
4. Type ``/plan act`` to build the latest plan, or ``/plan off`` to leave without building.

Tip

``/new`` clears the saved plan. Plan commands need an idle agent with no running or queued work.

8. Skills and SkillPlan

Using skills

Skills are reusable instruction bundles, for example a team review checklist. Turn them on per tab with ``/skill``. Only selected skills are shared with the assistant. You can also run ``/skill name1,name2`` directly, or ``/skill none`` to clear them.

- Type ``/skill`` to see names plus one-line summaries. Type to filter, Up/Down or PgUp/PgDn to move, Space to toggle, Enter to apply, Esc to leave, Ctrl+C to cancel.
- Your account, workspace, and configured folders can each provide skills.
- New files appear in ``/skill`` without restarting.
- A ready-made collection ships with qcode under docs/skills. Enable only what helps.

Creating a skill safely

Use ``/skillplan`` alone or with a rough idea to turn it into a reviewed skill. It asks when the skill applies, its workflow, inputs and outputs, constraints, validation, name, and scope, with follow-ups when an answer leaves a decision open.

```
/skillplan review database migrations before deployment
```

- ``/skillplan show`` reviews the draft. ``/skillplan create`` writes it. ``/skillplan off`` leaves without writing.
- Names allow 1-64 lowercase letters, digits, hyphens, or underscores; files must fit 64 KiB. qcode never overwrites an existing SKILL.md.
- After creation, run ``/skill`` to enable the new skill.
- SkillPlan can read context but cannot run shell commands or change files until you approve creation.

9. Remembering preferences with /learn

Use `/learn` after a good answer to save a durable preference or reusable procedure. qcode shows a preview and asks `Apply these global learning changes? [y/N]`. Nothing is saved unless you answer `y` or `yes`; Enter, `n`, or Ctrl+C cancels.

Command	What it does for you
<code>/learn</code>	Propose memory from recent conversation.
<code>/learn list</code>	Show saved items and IDs.
<code>/learn forget <id></code>	Review then delete one item.
<code>/learn compact</code>	Merge duplicates (asks first, backs up).

- Memory is per user on this device and shared across folders.
- Keep entries reusable: state the language or condition when they apply.
- Review proposals: secrets are rejected, but you decide what is worth keeping.

10. Web fetch and search

Web tools read public pages and search results as text. They start OFF. Turn them on for the current session with ``/tool``. The backend setting alone does not enable them.

- Type ``/tool`` and enable ``web_fetch`` and/or ``web_search`` independently.
- ``/new`` or restarting turns them off again. One-shot runs keep them off.
- Use them for docs and public references. They do not run page scripts and do not open a browser.

Tip

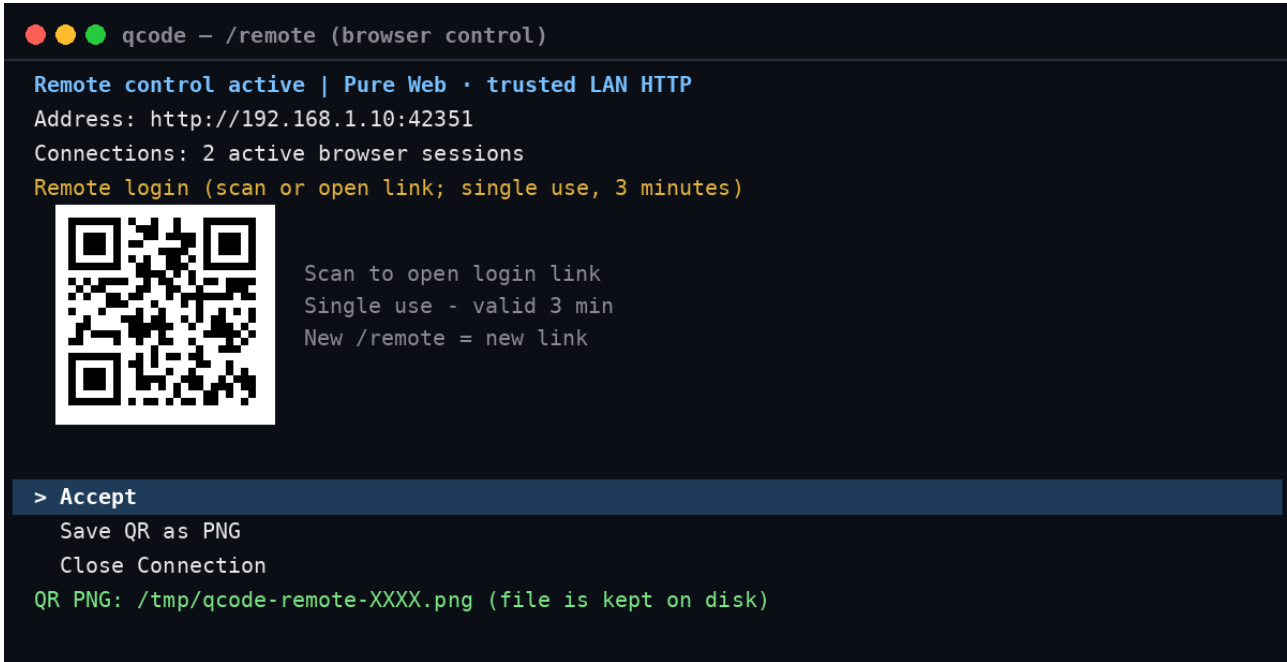
If a page needs login or runs an app, fetch will not see it. Paste the relevant text into qcode instead.

11. Sessions: autosave, resume, and export

- Sessions autosave about every two seconds, after agent work and commands, and on clean exit or session switch. Each launch starts its own session.
- Type `/resume`` to reopen a session for this folder. Entries show a preview and age, newest first. Finish or cancel running work before switching.
- Empty and demo/one-shot runs are not saved. `/new`` resets only the current tab inside its session.
- After a crash, qcode reopens the last good checkpoint and marks unfinished work interrupted. It never reruns tools by itself.
- Sessions live outside your project: `~/.local/state/qcode/sessions` on Linux (or `$XDG_STATE_HOME`), `~/Library/Application Support` on macOS, `%AppData%` on Windows. They keep private file permissions.

12. Controlling qcode from a browser

Type ``/remote`` to drive the same session from a phone or another browser. Pick Pure Web for a trusted local network or Tailscale for your tailnet (browsers must belong to the same tailnet). Avoid the ``No auth`` option unless you truly want an open short-lived demo.



```

qcode - /remote (browser control)

Remote control active | Pure Web · trusted LAN HTTP
Address: http://192.168.1.10:42351
Connections: 2 active browser sessions
Remote login (scan or open link; single use, 3 minutes)

[QR Code]

Scan to open login link
Single use - valid 3 min
New /remote = new link

> Accept
  Save QR as PNG
  Close Connection
QR PNG: /tmp/qcode-remote-XXXX.png (file is kept on disk)

```

Figure 8: `/remote` screen. Scan the code or open the link, then Accept. Save QR as PNG if the code is too tall for your window.

1. Type ``/remote`` and choose how to connect. For multiple networks, pick the interface to share.
 2. Scan the QR or open the shown link on the other device. Each link is single-use and lasts 3 minutes; issuing a new link invalidates a previous unredeemed one, while existing browsers stay connected.
 3. If the code does not fit, choose ``Save QR as PNG`` and open the shown file path (in WSL, convert it with ``wslpath -w <path>``). The file is kept on disk.
 4. Choose Accept to keep the connection open. Run ``/remote`` again for one more device; existing browsers stay connected.
 5. To stop all browsers, choose Close Connection and confirm, or quit qcode. Used and expired links tell you to run ``/remote`` again.
- Browser Enter and Steer update the busy task; Queue submits a separate task. Tab navigates controls normally.
 - Remove cancels pending input; Cancel active work stops the running task. Rejected submissions keep your draft.
 - Reloading the same browser tab resumes. A fresh browser needs a fresh link.
 - In the browser, ``/export`` downloads the same HTML you get in the terminal.
 - Use only on networks you trust. Pure Web is plain HTTP on your LAN.

13. Settings you actually change

Most daily choices live in the terminal (``/model``, ``/maxsteps``, ``/statusline``, ``/theme``, ``/tool``). Use a config file only for defaults you always want.

Terminal colors and workspace visibility

Use `/theme` to preview terminal palettes with Up/Down. Enter applies and saves the choice; Esc or Ctrl+C cancels. Default (Auto) follows your terminal. Dark and light choices include Catppuccin, Dracula/Alucard, Gruvbox, Solarized, and Nord. The setting is terminal-wide and saves from any agent tab; it does not change browser colors.

WS uses available status bar space and keeps the final two complete folders when possible, using a second row as needed. Home-relative paths retain `~/`; omitted parents appear as an ellipsis folder, for example `~/.../work/foo`. If the final folder name must be clipped, its ending gets an ellipsis: `~/workspace/this-folder-is...` (the final dots shorten the name).

```
provider = "ollama"
model = "qwen2.5-coder:7b"
max_steps = 32
theme = "catppuccin-mocha"
sandbox = true
```

- Copy ``config.toml.example`` to your user config location; missing files are ignored. Priority, highest first: file beside the program, user config (`~/local/etc/qcode` on Linux, `~/Library/Application Support` on macOS, `%AppData%` on Windows), system config (`/usr/local/etc/qcode`), then `etc/` beside the program.
- Command flags beat environment variables. Both beat config files. Config files beat built-ins. ``--api-key`` beats config and environment keys.
- Prefer environment variables for keys: `QCODE_API_KEY` or `OPENAI_API_KEY`. Keep key files private (mode 0600).
- Common knobs: `thinking_level`, `agent_timeout` like "5m", `interactive true/false`, `statusline_hidden`, `web_search backend`, `learning budget`, `skills paths`.
- Restart after changing the web backend. ``/model`` and ``/maxsteps`` on ``main`` save automatically.

14. Troubleshooting and tips

Command	What it does for you
No colors?	Run with NO_COLOR=1. For ASCII borders use QCODE_ASCII=1.
Narrow window?	The path shortens first; the bar wraps to a second left-aligned line. Low-priority parts drop only if two lines still overflow. Hide more with /statusline.
Slow session?	Try /compact, then /new for a fresh tab context.
Need detail?	Try /verbose. For step limits use /maxsteps N.
Lost output?	Scroll with PageUp/PageDown. /history finds old answers. /export saves HTML.
Remote blocked?	On Linux run once: sudo tailscale set --operator=\$USER. Then retry /remote.

Tip

Good defaults for most users: local model for drafts, stronger hosted model for final edits, Plan mode for risky changes, one agent per independent job.

15. Cheat sheet (copy this page)

Command	What it does for you
/help	Help for all or one command.
/bash <cmd>	Run a shell command.
/model	Switch model by name only (+ thinking).
/agent ...	Create, list, switch, rename, cancel, close.
/plan ...	Plan, show, act, off.
/skill	Enable instruction bundles.
/skillplan ...	Draft, show, create, or leave skill design.
/interactive	Let normal work ask up to 3 questions per prompt.
/learn ...	Save/list/forget/compact preferences.
/tool	Toggle web fetch/search.
/resume, /new	Reopen session / reset tab.
/history	Find and re-read answers.
/diff	Expand saved diffs.
/compact	Summarize old context.
/verbose	Show/hide detailed traces.
/export	Save pretty or raw HTML.
/remote	Browser control via QR link.
/statusline	Choose status bar parts.
/theme	Preview and save terminal colors.
/maxsteps	Show/set step limit.
/clear	Clear view and redraw header.
/quit, /exit	Save session and exit.

One-shot recipes:

```
qcode --model qwen2.5-coder:7b "explain this repository"
qcode --provider openai --model gpt-5 "summarize the diff"
qcode --demo "show me how qcode works"
```

About this manual

This manual teaches use, not construction. It omits source layout, network internals, and developer extension points on purpose. For command details, type `/help`` inside qcode. Generated September 30, 2026 as version dev. Screenshots are illustrative examples rendered for print clarity.

Chapters in this manual

- How to use this manual
- 1. Getting started
- 2. Touring the terminal
- 3. Asking, queueing, and history
- Use cases: understand and fix; plan and divide work; reuse and continue
- 4. Slash commands at a glance
- 5. Choosing models and providers
- 6. Working with multiple agents
- 7. Planning before coding
- 8. Skills and SkillPlan
- 9. Remembering preferences with `/learn`
- 10. Web fetch and search
- 11. Sessions: autosave, resume, and export
- 12. Controlling qcode from a browser
- 13. Settings you actually change
- 14. Troubleshooting and tips
- 15. Cheat sheet